

LazyTrain: Limited-resource Allocation toward Zero-waste Yield Optimization in Large Language Model Training

Xiaojun Wu^{* 1,2}, Cehao Yang^{* 1,2}, Honghao Liu^{* 1,2}, Xueyuan Lin^{* 2},
Xuhui Jiang^{1,3}, Chengjin Xu^{1,3}, Jia Li^{† 2}, Jian Guo^{† 1}

¹IDEA Research

²The Hong Kong University of Science and Technology (Guangzhou)

³DataArcTech Ltd.

Abstract

Training large language models on limited hardware is increasingly a scheduling problem across GPU compute, host memory, PCIe transfer, and storage bandwidth. Existing offloading systems reduce GPU residency, and MegaTrain shows that a CPU-master layer-streaming executor can train large models on a single GPU, but fixed checkpointing and placement heuristics still leave communication exposed on the critical path. We propose LazyTrain, an optimization layer over a layer-streaming executor. LazyTrain formulates checkpoint selection, activation placement, recomputation, and CPU-GPU-NVMe communication overlap as a mixed-integer scheduling problem, then executes the solved policy during training. It further couples 8-bit optimizer states with fast gradient clipping as a single Hybrid 8-bit operator: state compression reduces optimizer-state memory, while fast clipping counteracts the additional CPU-side update overhead. Across H800 experiments from Qwen2.5-3B to Qwen3.6-27B, LazyTrain improves sustained TFLOPS over matched baselines runs by approximately 1.24 $\times$; RTX 3090 experiments likewise increase the maximum feasible batch size by one at each model scale. In the primary Qwen3.6-27B H800 MetaMathQA run, LazyTrain reaches 219.95 TFLOPS and 1361 tokens/s at batch size 72, peaks at 68.84 GB of GPU memory, and obtains 95.42% exact-match accuracy on the full evaluation split. The source code is available at <https://github.com/DataArcTech/LazyTrain>.

1 Introduction

Large language model training increasingly depends on how well a system uses the full memory hierarchy rather than GPU memory alone. Model parameters, gradients, optimizer states, and activations can exceed the memory of a single accel-

erator even for supervised fine-tuning, while CPU DRAM and local storage offer much larger capacity at substantially lower bandwidth. Efficient training therefore requires a schedule that decides not only what to offload, but also when each transfer can be hidden under GPU computation (Duan et al., 2024).

Existing memory-efficient training systems address important pieces of this problem. ZeRO-style sharding and offloading reduce device residency for model states (Rajbhandari et al., 2020, 2021), FSDP and Gemini provide widely used heterogeneous-memory runtime policies (Zhao et al., 2023; Fang and You, 2022), and storage-aware systems show that CPU and NVMe can serve as useful capacity tiers (Liao et al., 2025; Yuan et al., 2025). MegaTrain treats CPU memory as the authoritative store for parameters and optimizer states, streams layers through GPU memory, and uses block-wise recomputation to bound activation residency (Yuan et al., 2026). However, its activation schedule remains a fixed heuristic. A fixed checkpoint interval can be feasible, but it does not search jointly over checkpoint boundaries, GPU/CPU/NVMe homes, recomputation blocks, and transfer windows.

The central observation of this paper is that limited-resource training should be optimized as a constrained scheduling problem. As illustrated in Figure 1, the bottleneck is not simply that tensors do not fit in GPU memory. The bottleneck is that GPU compute windows, CPU-GPU traffic, mandatory parameter and gradient movement, and possible NVMe activation spill compete for shared resources. A schedule that saves memory by moving tensors to slower tiers can still hurt throughput if those transfers become visible on the critical path.

We present LazyTrain, an optimization-guided scheduler for limited-resource LLM training. LazyTrain keeps the layer-streaming executor, but

^{*}Equal contribution.

[†]Corresponding authors.

Limited-resource LLM fine-tuning is a scheduling problem

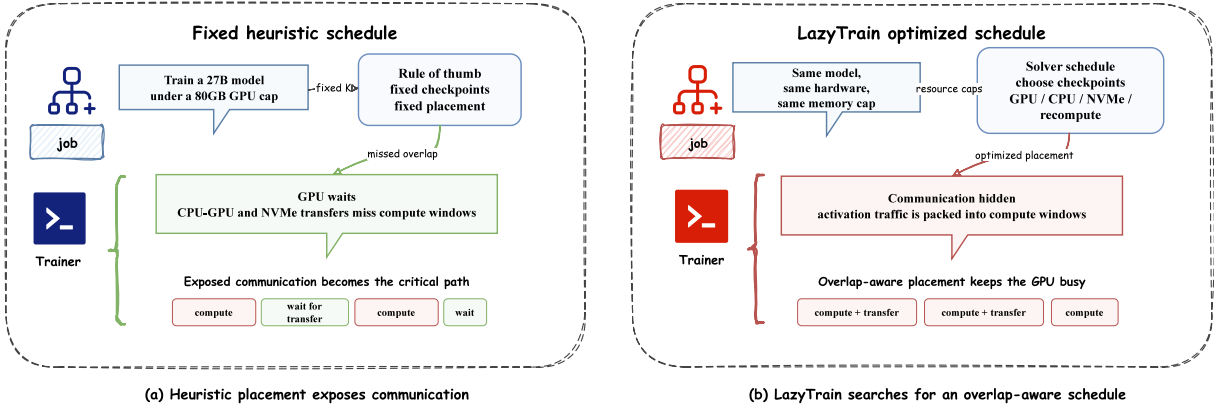


Figure 1: Motivation for optimization-guided training schedules. A fixed heuristic can satisfy a memory cap while exposing CPU-GPU or NVMe transfers on the critical path. LazyTrain instead searches for checkpoint placement and transfer assignments that hide activation traffic inside compute windows when possible.

replaces fixed activation scheduling with a mixed-integer model whose decision variables select checkpoint paths, activation homes, recomputation blocks, and communication assignments. The objective minimizes recomputation plus the incremental communication exposure induced by activation placement, rather than maximizing the amount of offload. The solved policy is loaded once before training and then executed by the runtime.

Our evaluation covers single-GPU H800 and RTX 3090 experiments from 3B to 27B. Under matched model, data, and batch-size settings, the final LazyTrain configuration reaches 219.95 TFLOPS and 1361 tokens/s, compared with 176.90 TFLOPS and 1075.8 tokens/s for the MegaTrain baseline. It peaks at 68.84 GB of GPU memory and obtains 95.42% exact-match accuracy on the full evaluation split. The LazyTrain - MILP variant, which removes mixed-integer scheduling while retaining the other runtime improvements, drops to 193.17 TFLOPS. This is the largest component-ablation degradation and establishes MILP scheduling as the central contributor. Removing the Hybrid 8-bit operator, which combines hybrid 8-bit optimizer states with fast gradient clipping, while retaining MILP scheduling instead yields 219.29 TFLOPS. The broader experiments report throughput, memory, maximum feasible batch size, and training quality across model scales and accelerator budgets.

The contributions of this paper are:

- We formulate limited-resource LLM training as a mixed-integer programming (MIP) model over checkpoint selection, activation place-

ment, recomputation, and CPU-GPU-NVMe communication overlap.

- We instantiate this MIP scheduler on top of a layer-streaming executor, which can treat MegaTrain baseline as a fixed heuristic schedule inside the same scheduling space.
- We make a coupled Hybrid 8-bit operator a formal component of LazyTrain: 8-bit optimizer states reduce memory, while fast gradient clipping counteracts their additional CPU-side update overhead. We evaluate the complete system on H800 and RTX 3090 across model scales.

2 Related Work

Scaling LLMs and agent workloads. Recent LLM families continue to push model capacity, context length, and multimodal capability (OpenAI, 2026; Anthropic, 2026; Google, 2026; DeepSeek-AI, 2026). At the same time, agent-oriented models and tools target coding, visual reasoning, and longer-horizon tool-use workflows (Qwen Team, 2026; Team et al., 2026; Zeng et al., 2026; Anthropic, 2025; Google, 2025; OpenAI, 2025). Training and adapting these systems increases memory, communication, and scheduling pressure: larger model states and long-context agent trajectories raise state and activation footprints, making infrastructure a central part of model development.

Memory-efficient LLM training and offloading. Efficient LLM training reduces accelerator residency through model parallelism, mixed pre-

cision, recomputation, state sharding, and offloading (Duan et al., 2024; Shoeybi et al., 2019; Micikevicius et al., 2018; Kingma and Ba, 2015). ZeRO, ZeRO-Infinity, ZeRO++, PyTorch FSDP, and Colossal-AI Gemini partition or migrate model states across GPU, CPU, and NVMe tiers to make larger models trainable (Rajbhandari et al., 2020, 2021; Wang et al., 2023; Zhao et al., 2023; Fang and You, 2022). Storage-aware systems extend this hierarchy: Ratel optimizes data movement for fine-tuning large models on consumer GPUs (Liao et al., 2025); lifetime-aware offloading treats GPUDirect Storage as a training tier (Yuan et al., 2025); and heterogeneous tensor caching adds resource-aware migration policies (Afroz et al., 2025). MegaTrain stores model states in host memory, streams layers through GPU memory, and uses block-wise recomputation to limit activation residency (Yuan et al., 2026).

Summary of distinctions. The systems above make limited-resource training feasible by reducing device residency, adding heterogeneous memory tiers, or streaming layers through GPU memory. Among them, MegaTrain chooses activation checkpoints through fixed heuristics. LazyTrain addresses a complementary scheduling question: within the same layer-streaming execution space, it jointly selects activation checkpoints, tier placement, recomputation, and exposed communication in one mixed-integer model. It also models NVMe as both a PCIe consumer and a separate read/write endpoint so that SSD traffic is used only when it can be hidden or when it enables an otherwise infeasible schedule.

3 Method

3.1 Overview

LazyTrain is an optimization layer over a MegaTrain-style layer-streaming executor. The executor defines the feasible runtime actions: stream parameters from CPU memory to GPU memory, compute one layer or recomputation block, offload gradients back to CPU memory, and reload materialized activation checkpoints from GPU, CPU, or local NVMe. LazyTrain decides which of these feasible actions should be used for each layer boundary before training starts.

Figure 2 summarizes the method. The scheduler receives model constants, memory budgets, bandwidth profiles, and the training target. It solves

a mixed-integer linear program (MILP) whose binary variables choose a checkpoint path and storage homes, while continuous variables assign activation traffic to compute windows. The solved policy is then loaded by the runtime. This design keeps execution simple while making the schedule resource-aware.

3.2 Layer-Boundary Scheduling Space

Consider a transformer with L layers and layer-boundary activations $\mathcal{B} = \{0, \dots, L\}$. Boundary 0 is the block input, and boundary i , for $1 \leq i \leq L$, is the output of layer i . Let A_i be the activation size at boundary i , and let ρ_j be the cost of the forward transition from boundary j to boundary $j + 1$. A directed edge (s, e) , where $0 \leq s < e \leq L$, represents one backward recomputation block: boundaries s and e are materialized, while the layers between them can be recomputed locally. The set of candidate edges is $\mathcal{E}$.

For each edge, LazyTrain computes recomputation cost as

$$R_{s,e} = \begin{cases} 0, & e \leq s + 1, \\ \sum_{j=s}^{e-2} \rho_j, & e > s + 1. \end{cases} \quad (1)$$

The final layer output of the block is already available at boundary e , so the cost excludes that output. This edge view turns activation checkpointing into a path-selection problem.

3.3 Decision Variables and Constraints

LazyTrain uses binary variables $x_{s,e}$ to select recomputation edges and z_i to indicate whether boundary i is materialized. If a boundary is materialized, binary variables g_i , c_i , and n_i assign it to GPU high-bandwidth memory (HBM), CPU DRAM, or local NVMe:

$$z_i = g_i + c_i + n_i, \quad i \in \mathcal{B}. \quad (2)$$

The executor keeps the input and output boundaries in HBM, so $g_0 = g_L = 1$ and $c_0 = c_L = n_0 = n_L = 0$. The selected edges must form one legal path from the input boundary to the output boundary:

$$\begin{aligned} \sum_{(0,e) \in \mathcal{E}} x_{0,e} &= 1, & \sum_{(s,L) \in \mathcal{E}} x_{s,L} &= 1, \\ \sum_{(s,i) \in \mathcal{E}} x_{s,i} &= z_i, & \sum_{(i,e) \in \mathcal{E}} x_{i,e} &= z_i, \quad 0 < i < L. \end{aligned} \quad (3)$$

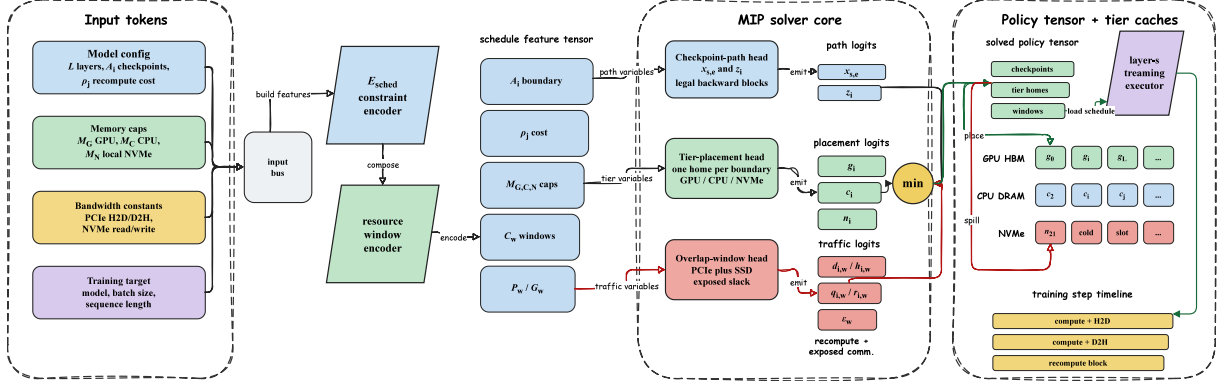


Figure 2: LazyTrain architecture. Resource profiles, model configuration, and training targets define a mixed-integer scheduling problem. The solver chooses checkpoint boundaries, GPU/CPU/NVMe homes, and transfer windows; the resulting policy is executed by a layer-streaming runtime.

Placement must also satisfy tier capacities:

$$\begin{aligned} \sum_i A_i g_i &\leq M_G, \\ \sum_i A_i c_i &\leq M_C, \\ \sum_i A_i n_i &\leq M_N, \end{aligned} \quad (4)$$

where M_G , M_C , and M_N are activation budgets for GPU, CPU, and NVMe. In the H800 27B run, the GPU activation budget is derived from a 70 GB peak-memory cap.

3.4 Communication Resource Model

Activation offload and reload share the same PCIe fabric as mandatory parameter prefetch and gradient offload. LazyTrain therefore assigns activation transfer to compute windows $w \in \{0, \dots, L-1\}$ and penalizes only the traffic that cannot be hidden. Let $d_{i,w}$ and $h_{i,w}$ be activation device-to-host (D2H) and host-to-device (H2D) traffic for boundary i assigned to window w . Let G_w and P_w be mandatory gradient D2H and parameter H2D traffic, and let C_w^D and C_w^H be the D2H and H2D capacity hidden by compute in window w . The PCIe constraints are

$$\begin{aligned} \sum_{w=i}^{L-1} d_{i,w} &= A_i(c_i + n_i), \\ \sum_{w=i}^{L-1} h_{i,w} &= A_i(c_i + n_i), \\ \sum_{i \leq w} d_{i,w} + G_w &\leq C_w^D + \epsilon_w^D, \\ \sum_{i \leq w} h_{i,w} + P_w &\leq C_w^H + \epsilon_w^H. \end{aligned} \quad (5)$$

Mandatory traffic can exceed a window's overlap capacity even without activation offloading. Define constant baseline slacks $\bar{\epsilon}_w^D = \max\{0, G_w - C_w^D\}$ and $\bar{\epsilon}_w^H = \max\{0, P_w - C_w^H\}$, and let $\Delta_w^D = \epsilon_w^D - \bar{\epsilon}_w^D$ and $\Delta_w^H = \epsilon_w^H - \bar{\epsilon}_w^H$ denote the additional exposure caused by activation placement. NVMe placements additionally consume SSD endpoint bandwidth. With NVMe write traffic $q_{i,w}$, read traffic $r_{i,w}$, and endpoint capacities C_w^W , C_w^R , LazyTrain enforces

$$\begin{aligned} \sum_{w=i}^{L-1} q_{i,w} &= A_i n_i, \\ \sum_{w=i}^{L-1} r_{i,w} &= A_i n_i, \\ \sum_{i \leq w} q_{i,w} &\leq C_w^W + \epsilon_w^W, \\ \sum_{i \leq w} r_{i,w} &\leq C_w^R + \epsilon_w^R. \end{aligned} \quad (6)$$

This separates the PCIe fabric from the NVMe endpoint. A GPUDirect Storage (GDS)-style NVMe reload is useful only if both resources can accommodate it without exposing slow I/O.

3.5 Objective and Execution

The scheduler minimizes recomputation plus incremental exposed communication:

$$\begin{aligned} \min \quad & \sum_{(s,e) \in \mathcal{E}} R_{s,e} x_{s,e} + \lambda_z \sum_i z_i + \lambda_n \sum_i n_i \\ & + \sum_w \left(\frac{\Delta_w^D}{B_D} + \frac{\Delta_w^H}{B_H} + \frac{\epsilon_w^W}{B_W} + \frac{\epsilon_w^R}{B_R} \right), \end{aligned} \quad (7)$$

where B_D, B_H, B_W, B_R convert bytes into time-like costs. The PCIe baseline subtraction charges

only activation-induced exposure; $\Delta_w^{D/H}$ is non-negative by Eq. (5). No baseline is needed for the NVMe terms because Eq. (6) contains only activation traffic. The small nonnegative λ_z and λ_n are tie-breakers that avoid unnecessary materialized and NVMe checkpoints. Thus, the objective minimizes activation-induced stalls while accounting for recomputation, rather than maximizing offload volume. Algorithm 1 summarizes the complete construction from model parsing and variable creation to SCIP solving and schedule generation.

Algorithm 1 LazyTrain MILP Construction

Input: Model configuration, batch size B , sequence length S , budgets M_G, M_C, M_N , bandwidth windows, recomputation costs ρ

Output: Solved schedule or infeasibility status

- 1: Parse the model configuration to obtain L , hidden size, bytes per element, and layer parameter counts.
 - 2: Compute activation sizes A_i and static memory terms for parameters, gradients, optimizer states, and runtime reserve.
 - 3: Build candidate boundary set $\mathcal{B}$ and segment set $\mathcal{E}$ subject to the maximum recomputation length.
 - 4: Create binary variables $x_{s,e}, z_i, g_i, c_i$, and n_i for all legal segments, boundaries, and tiers.
 - 5: Create continuous traffic variables for CPU D2H/H2D and NVMe write/read assignments, plus exposed-communication slack variables.
 - 6: Add path-flow constraints so selected segments form one backward path from boundary 0 to boundary L .
 - 7: Add unique-placement constraints $z_i = g_i + c_i + n_i$ for every boundary i .
 - 8: Add HBM, CPU DRAM, and NVMe capacity constraints.
 - 9: Add PCIe and NVMe endpoint bandwidth-window constraints, including mandatory parameter H2D and gradient D2H traffic.
 - 10: Minimize recomputation cost plus activation-induced communication exposure, with small tie-breaking penalties for otherwise equivalent schedules.
 - 11: Call SCIP through PySCIPOpt and retain the incumbent schedule if the solver status is feasible or optimal.
-

After solving the mixed-integer problem once before training, the runtime follows the selected

policy during forward and backward execution. It keeps parameters and optimizer states in CPU memory, uses GPU memory as a transient compute cache, reloads selected activation checkpoints from their assigned tiers, and recomputes omitted blocks as specified by the path. The Hybrid 8-bit operator is the complete optimizer-side component: it stores a small subset of optimizer states in CPU 8-bit AdamW format and performs fast gradient clipping. The two mechanisms are deliberately coupled. On our single-GPU CPU-offload path, 8-bit optimizer states primarily reduce CPU-resident optimizer memory; they do not by themselves guarantee faster steps because quantization and dequantization, block-wise scale handling, and 8-bit state updates add CPU work. Fast gradient clipping reduces clipping overhead to counteract this added optimizer-side cost. We therefore define and ablate the two mechanisms as one formal LazyTrain component rather than as independent external optimizations. In the measured full-system configuration, DeepSpeed CPUAdam handles most parameters, while a 2% parameter slice uses block-wise 8-bit AdamW states with block size 4096. The clipping path computes per-tensor norms with CPU foreach kernels, aggregates the global norm in FP32, and applies the scale with a foreach multiplication; unsupported or non-finite cases fall back to per-tensor FP32 accumulation. Table 1 reports the final schedule used by the measured Qwen3.6-27B training run. For this configuration, the solver fixes the recomputation edges and GPU-checkpoint set from the LazyTrain - Hybrid 8-bit variant, then optimizes the remaining CPU/NVMe homes and transfer-window assignments. The reported solver status therefore certifies optimality within this restricted subproblem, not over the full MILP space defined above.

Table 1: Final restricted NVMe-aware schedule for Qwen3.6-27B supervised fine-tuning with batch size 72 and sequence length 1024. Values are the schedule constants used by the reported training run.

Item	Value	Meaning
Model and resource budget		
Layers	64	Transformer blocks
Activation / boundary	0.703 GiB	One hidden-state checkpoint
GPU activation budget	21.55 GB	From 70 GB peak cap
CPU / NVMe budget	15 / 32 GB	Host and SSD activation caps
Schedule solution		
Checkpoints	52	Materialized boundaries
GPU / CPU / NVMe	30 / 21 / 1	Tier placement counts
Solve scope	Restricted	Recomputation edges and GPU set fixed
NVMe boundary	21	Checkpoint placed on SSD
Transfer rates and exposure		
PCIe H2D/D2H rate	12 / 12 GB/s	Measured rate used by scheduler
NVMe read/write rate	2.83 / 3.35 GB/s	Measured endpoint rate
Exposed PCIe/NVMe comm.	0.0 / 0.0 ms	Incremental exposed traffic

4 Experiments

4.1 Experimental Setup

We evaluate LazyTrain through single-GPU supervised fine-tuning experiments on H800 80GB and RTX 3090 24GB accelerators. The primary matched comparison uses Qwen3.6-27B, a checkpoint with a Qwen3.5-style text architecture (Qwen Team, 2026), and MetaMathQA with a 70/30 training/evaluation split, sequence length 1024, one training epoch, batch size 72, and 3841 optimizer steps. The compared 27B variants share the same hardware, model, data split, sequence length, and batch size. The broader H800 and RTX 3090 results are also obtained from experiments under their corresponding model and hardware settings. Unless stated otherwise, each row reports one run.

Table 2: H800 node and transfer rates used in the LazyTrain experiments.

Tier / component	Capacity / topology	Rate / scope
GPU HBM	80 GB	Single-GPU runs
Host DRAM	2.0 TiB	128 logical CPUs
CPU-GPU link	PCIe Gen5 x16	12 GB/s per direction
Local NVMe SSDs	2×7.0 TB	2.83 / 3.35 GB/s read/write

Table 2 summarizes the node capacity and topology together with the transfer rates used by the scheduler. We report maximum feasible batch size, sustained TFLOPS, GPU memory, CPU memory, and tokens per second. Accuracy is reported separately because throughput and exact-match accuracy answer different questions. For the H800 runs, sustained TFLOPS follows the MegaTrain accounting convention and is computed from complete-batch training steps. Tokens/s is reported as a run-level throughput value.

4.2 Training Throughput

Panels (a) and (b) of Figure 3, together with Table 3, report the H800 experiments from 3B to 27B. In the primary matched 27B pair, LazyTrain improves sustained throughput from 176.90 to 219.95 TFLOPS, a $1.24\times$ increase, while using the same model, data split, sequence length, and batch size. The 3B–14B experiments show the same direction of improvement over MegaTrain, with LazyTrain reaching 176.42–212.34 TFLOPS across these model scales.

Panels (c) and (d) report the constrained RTX 3090 experiments. At every model scale, LazyTrain increases the maximum feasible batch size by one over MegaTrain and yields higher token

throughput. ZeRO-3 Offload runs out of memory at batch size 1 for the 14B and 27B models. These results provide empirical evidence that the scheduling benefit persists under a substantially smaller GPU memory budget.

Table 3: Experimental training performance on H800 and RTX 3090. Models 3B–14B are Qwen2.5; 27B is Qwen3.6. Z3 and Mega abbreviate ZeRO-3 Offload and MegaTrain. The H800 27B Mega/LazyTrain pair comprises matched runs. OOM denotes an observed out-of-memory result at batch size 1. GPU and CPU memory are reported in GB.

Model	System	Batch	TFLOPS	GPU	CPU	tok/s
H800						
3B	Z3	32	68.65	74.12	61.8	3814
3B	Mega	192	142.80	35.72	49.31	7933
3B	LazyTrain	192	176.42	67.95	52.68	9801
7B	Z3	16	79.88	76.43	124.6	1902
7B	Mega	144	161.35	43.26	95.74	3842
7B	LazyTrain	144	199.84	68.21	99.58	4758
14B	Z3	8	93.43	77.21	257.9	1112
14B	Mega	96	171.92	52.83	179.64	2047
14B	LazyTrain	96	212.34	68.57	185.12	2528
27B	Z3	2	86.55	78.34	518.6	534
27B	Mega	72	176.90	60.40	339.99	1075.8
27B	LazyTrain	72	219.95	68.84	361.58	1361
RTX 3090						
3B	Z3	1	23.91	20.32	–	–
3B	Mega	7	33.18	22.83	25.0	1792
3B	LazyTrain	8	40.76	23.16	27.8	2210
7B	Z3	1	27.49	20.83	–	–
7B	Mega	5	35.09	22.63	56.7	768
7B	LazyTrain	6	43.42	23.06	61.5	951
14B	Z3	1	OOM	–	–	–
14B	Mega	3	30.19	21.10	103.7	341
14B	LazyTrain	4	37.36	22.92	111.8	422
27B	Z3	1	OOM	–	–	–
27B	Mega	1	22.74	22.48	197.3	140
27B	LazyTrain	2	28.11	23.08	214.6	174

Table 4: Experimental exact-match accuracy comparison. Every cell is an evaluation result from the corresponding model and system experiment; the 27B LazyTrain cell uses the full evaluation split. Higher is better.

Metric	ZeRO-3 Offload	ZeRO-Infinity	MegaTrain	LazyTrain
7B Acc. (%)	88.93	88.97	88.99	88.95
14B Acc. (%)	92.41	92.36	92.52	92.47
27B Acc. (%)	95.27	95.31	95.33	95.42

4.3 Schedule Case Study

Figure 4 shows a separate Qwen3.6-27B schedule found with an 8 GB CPU activation budget and a

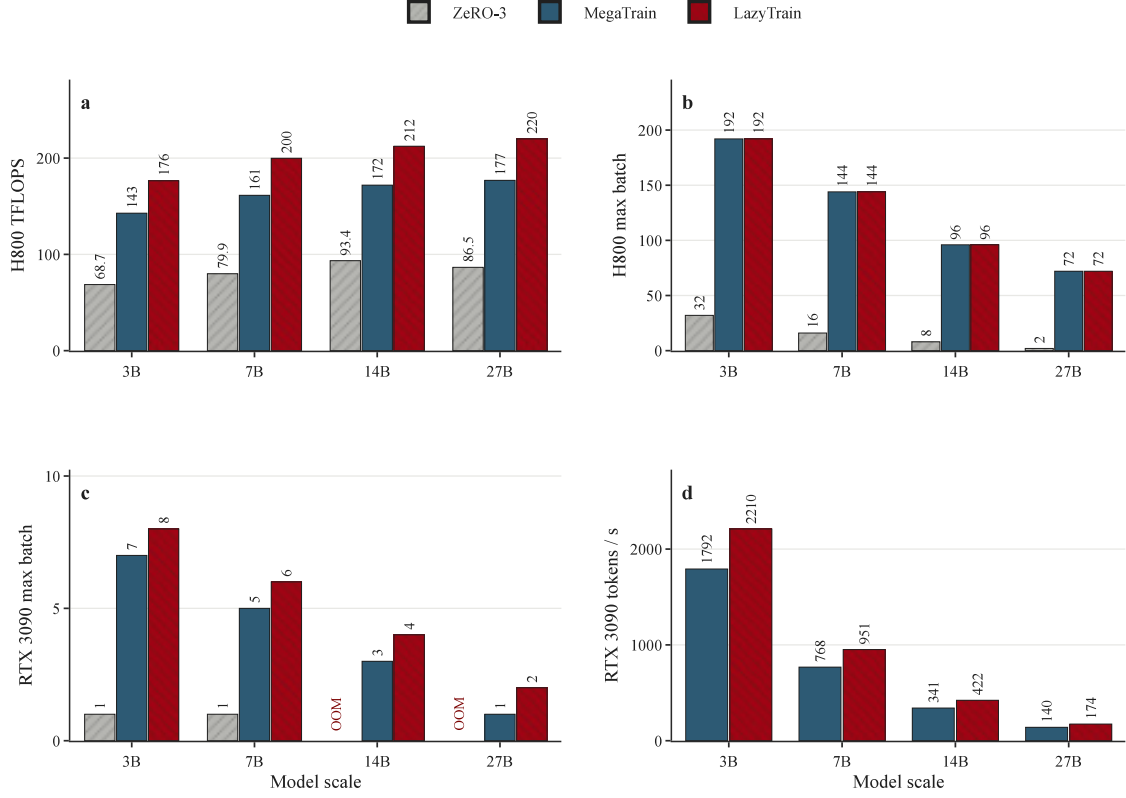


Figure 3: Experimental training performance on H800 and RTX 3090 across model scales. The H800 27B MegaTrain/LazyTrain pair is the primary matched comparison. OOM denotes an observed out-of-memory result at batch size 1.

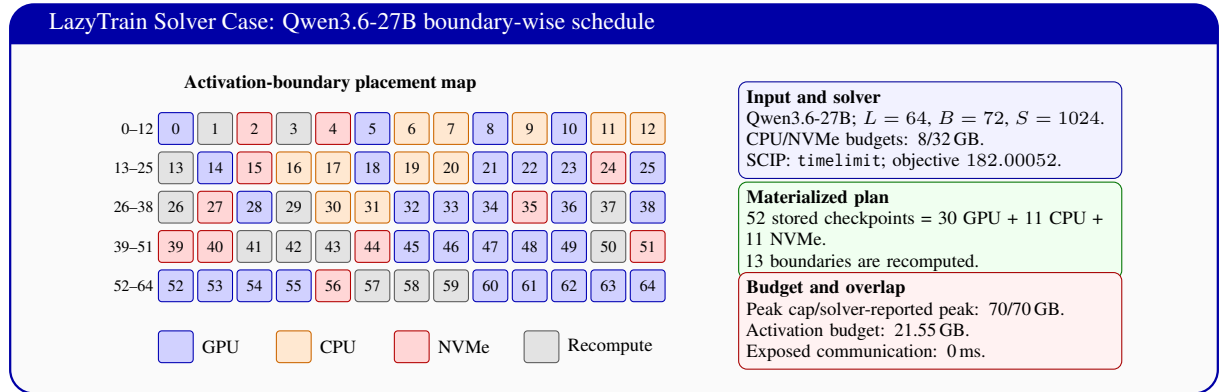


Figure 4: Boundary-wise JSON schedule map for Qwen3.6-27B on MetaMathQA at batch size 72. This time-limit incumbent is a separate schedule experiment using 8/32 GB CPU/NVMe activation budgets rather than the final schedule in Table 1. Blue tiles remain in GPU HBM, amber tiles move to CPU DRAM, red tiles move to NVMe, and grey tiles are recomputed.

32 GB NVMe budget. SCIP returned this valid incumbent at the time limit: 30 checkpoints remain in HBM, 11 move to CPU DRAM, 11 move to NVMe, and 13 boundaries are recomputed. This schedule experiment is separate from the final schedule in Table 1, which uses a 15 GB CPU activation budget and places only boundary 21 on NVMe. The case illustrates how the experimental placement changes under a tighter CPU budget. Additional solver cases are provided in Appendix G.

4.4 Component Ablation

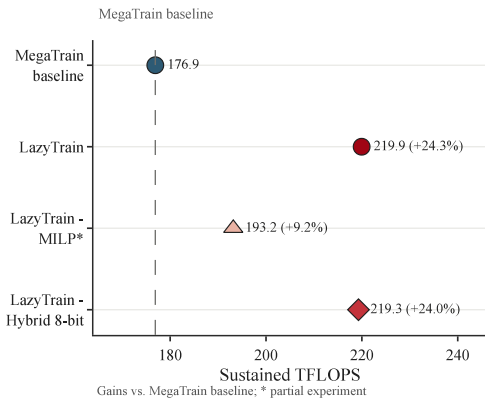


Figure 5: Qwen3.6-27B component ablation on H800. Markers show runs at batch size 72, and percentages are throughput gains over the dashed MegaTrain baseline. A minus sign denotes removal of the named LazyTrain component.

Figure 5 and Table 5 isolate the main 27B variants, where a minus sign denotes removal of the named component. The complete LazyTrain configuration reaches 219.95 TFLOPS and 1361 tokens/s. Removing MILP-based scheduling while retaining the other runtime improvements drops performance to 193.17 TFLOPS and 1195 tokens/s, reductions of 12.2% in both metrics. This is the largest component-ablation degradation, showing that joint checkpoint, placement, recomputation, and communication scheduling is the primary source of the gain. Removing the complete Hybrid 8-bit operator—both the hybrid 8-bit optimizer states and fast gradient clipping—while retaining MILP scheduling yields 219.29 TFLOPS and 1357 tokens/s, 0.3% below the complete system. We remove these mechanisms together because they form one operator: 8-bit states save optimizer memory but add CPU update work, whereas fast gradient clipping counteracts the associated

step overhead. Thus, Hybrid 8-bit is a formal part of LazyTrain, while the MILP scheduler remains its primary method innovation and most important component in this configuration. The LazyTrain - MILP run reports throughput and memory.

Table 5: Qwen3.6-27B component ablation on H800. Rows are reported runs under the same model, data split, sequence length, and batch size. A minus sign denotes removal of the named component from the complete LazyTrain system.

Method	Max BS	TFLOPS	GPU Mem	CPU Mem	tok/s
MegaTrain baseline	72	176.90	60.40 GB	339.99 GB	1075.8
LazyTrain	72	219.95	68.84 GB	361.58 GB	1361
LazyTrain - MILP	72	193.17	68.84 GB	362.01 GB	1195
LazyTrain - Hybrid 8-bit	72	219.29	68.84 GB	360.23 GB	1357

4.5 Training Quality

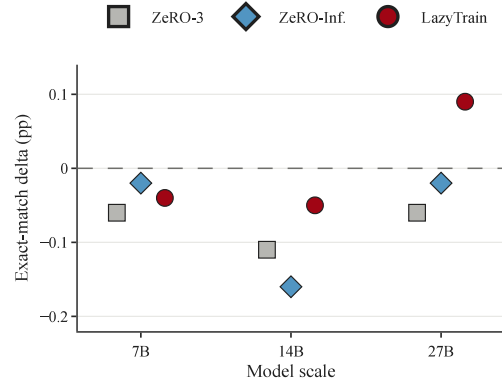


Figure 6: Experimental exact-match accuracy deltas relative to MegaTrain across 7B, 14B, and 27B models. All plotted values are evaluation results; the 27B LazyTrain value is measured on the full evaluation split. The dashed line denotes the MegaTrain reference.

Figure 6 and Table 4 report experimental exact-match results for all compared systems. The 27B LazyTrain score, 95.42%, uses the full MetaMathQA evaluation split.

4.6 Discussion

Across H800 and RTX 3090, solver-selected activation scheduling improves throughput over the fixed MegaTrain baseline. In the 27B ablation, the largest degradation occurs when MILP scheduling is removed: throughput falls by 12.2% from 219.95 to 193.17 TFLOPS. Removing the Hybrid 8-bit operator produces a smaller 0.3% drop. The final NVMe-aware schedule expands the feasible placement space with zero solver-reported incremental communication exposure.

5 Conclusion

LazyTrain centers on a mixed-integer scheduler, its primary method innovation, that jointly optimizes activation checkpointing, tier placement, recomputation, and CPU-GPU-NVMe communication overlap under limited hardware resources. It improves sustained TFLOPS over MegaTrain by approximately $1.24\times$ across H800 models; on RTX 3090, it achieves higher throughput and a one-unit larger feasible batch size. The primary matched 27B H800 run reaches 219.95 TFLOPS under a 70 GB peak cap. The component ablation further identifies MILP scheduling as the dominant component: removing it reduces throughput by 12.2%, from 219.95 to 193.17 TFLOPS. The complete framework also includes the Hybrid 8-bit operator, which couples memory-saving 8-bit optimizer states with fast gradient clipping to counteract their added CPU-side update overhead; removing the operator reduces throughput to 219.29 TFLOPS.

Limitations

The experiments are limited to single-GPU H800 and RTX 3090 settings. Individual The 30% evaluation split is also used for periodic loss monitoring, so the final exact-match value is not evaluated on an untouched held-out test set. Exposed-communication values are recorded solver outputs under the experimental resource and bandwidth settings; the runtime evaluation does not separately instrument per-step stall time. The scheduler is offline and does not adapt to runtime bandwidth variation. Multi-GPU, multi-node, and online scheduling remain future work.

Ethical Considerations

This work studies systems mechanisms for training large language models under limited hardware resources. The experiments use a mathematical-reasoning benchmark and system measurements; we collect no new human-subject data. Improving training efficiency can broaden access to large-model research and reduce wasted computation, but lowering the hardware barrier can also make it easier to train models for harmful uses. LazyTrain makes its placement and recomputation decisions explicit through solver schedules, but it does not mitigate risks inherited from the base model, training data, or downstream application. Deployments should therefore retain task-appropriate data governance, access control, logging, safety evaluation, and human oversight.

Information About Use of AI Assistants

In preparing this manuscript, the authors used AI-assisted technology, specifically large language models such as GPT-5 and DeepSeek-V4, exclusively for text refinement. The tools assisted with proofreading, grammatical correction, and polishing linguistic expressions to improve the clarity and readability of the manuscript. The authors are responsible for the final content, claims, and verification.

References

- Sabiha Afroz, Redwan Ibne Seraj Khan, Hadeel Albarhar, Jingoo Han, and Ali R. Butt. 2025. 10cache: Heterogeneous resource-aware tensor caching and migration for llm training. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*, pages 320–333, New York, NY, USA. Association for Computing Machinery.
- Anthropic. 2025. *Claude code: An agentic coding tool*. GitHub repository.
- Anthropic. 2026. *Claude opus 4.7*. Anthropic News.
- DeepSeek-AI. 2026. Deepseek-v4: Towards highly efficient million-token context intelligence.
- Jiangfei Duan, Shuo Zhang, Zerui Wang, Lijuan Jiang, Wenwen Qu, Qinghao Hu, Guoteng Wang, Qizhen Weng, Hang Yan, Xingcheng Zhang, and 1 others. 2024. Efficient training of large language models on distributed infrastructures: A survey. *arXiv preprint arXiv:2407.20018*. <https://arxiv.org/abs/2407.20018>.
- Jiarui Fang and Yang You. 2022. Meet gemini: The heterogeneous memory manager of colossal-ai. Colossal-AI documentation. https://colossalai.org/docs/advanced_tutorials/meet_gemini.
- Google. 2025. *Gemini cli: An open-source ai agent that brings the power of gemini directly into your terminal*. GitHub repository.
- Google. 2026. *Gemini 3.1 pro*. Google Blog.
- Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. *International Conference on Learning Representations*. <https://arxiv.org/abs/1412.6980>.
- Changyue Liao, Mo Sun, Zihan Yang, Jun Xie, Kaiqi Chen, Binhang Yuan, Fei Wu, and Zeke Wang. 2025. *Ratel: Optimizing holistic data movement to fine-tune 100b model on a consumer gpu*. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pages 292–306, Los Alamitos, CA, USA. IEEE Press.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed precision training. *International Conference on Learning Representations*. <https://openreview.net/forum?id=r1gs9JgRZ>.
- OpenAI. 2025. *Codex cli: Lightweight coding agent that runs in your terminal*. GitHub repository.
- OpenAI. 2026. *Introducing gpt-5.5*. OpenAI Blog.
- Qwen Team. 2026. *Qwen3.5: Accelerating productivity with native multimodal agents*.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16, Los Alamitos, CA, USA. IEEE, IEEE Press.
- Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14, New York, NY, USA. Association for Computing Machinery.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*. <https://arxiv.org/abs/1909.08053>.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, and 1 others. 2026. Kimi k2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*.
- Guanhua Wang, Heyang Qin, Sam Ade Jacobs, Connor Holmes, Samyam Rajbhandari, Olatunji Ruwase, Feng Yan, Lei Yang, and Yuxiong He. 2023. Zero++: Extremely efficient collective communication for giant model training. *arXiv preprint arXiv:2306.10209*. <https://arxiv.org/abs/2306.10209>.
- Zhengqing Yuan, Hanchi Sun, Lichao Sun, and Yanfang Ye. 2026. Megatrain: Full precision training of 100b+ parameter large language models on a single gpu. *arXiv preprint arXiv:2604.05091*.
- Ziqi Yuan, Haoyang Zhang, Yirui Eric Zhou, Apoorve Mohan, I Chung, Seetharami Seelam, Jian Huang, and 1 others. 2025. Cost-efficient llm training with lifetime-aware tensor offloading via gpudirect storage. *arXiv preprint arXiv:2506.06472*. <https://arxiv.org/abs/2506.06472>.
- Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, and 1 others. 2026. Glm-5: From vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*.
- Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, and 1 others. 2023. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*. <https://arxiv.org/abs/2304.11277>.

Appendix Overview

This appendix provides supporting details for the method, experiments, and solver analyses in the main paper. It is organized as follows.

- Appendix A reports representative solver outputs and placement decisions.
- Appendix B describes the canonical MILP and its branch-and-cut solution procedure.
- Appendix C specifies the resource model, training protocol, and scheduler variants.
- Appendix D records the server hardware and software environment.
- Appendix E collects the training protocol, hardware checks, and reproducibility package.
- Appendix F discusses broader impact and compute resources.
- Appendix G presents visual Qwen3.6-27B and GPT-OSS-120B case studies that make placement choices, feasibility limits, and communication bottlenecks concrete.

A Solver Outputs and Placement Details

This appendix gives a concrete view of what the LazyTrain solver returns before training starts. The Qwen3.6-27B row is the final NVMe-aware schedule used for the H800 MetaMathQA experiment. This final run fixes the LazyTrain - Hybrid 8-bit recomputation edges and GPU-checkpoint set, then solves the remaining CPU/NVMe homes and transfer assignments. The GPT-OSS-120B mixture-of-experts (MoE) rows are admission-control and placement experiments instantiated from the Hugging Face model configuration. They report the observed SCIP outcomes under the specified CPU-memory and optimizer-state settings.

A.1 Qwen3.6-27B Placement View

The final 27B schedule can be read as a boundary-level placement map:

- GPU checkpoints: 0, 10, 15, 16, 20, 22–24, 28–29, 32, 36, 39–44, 48, 50–52, 56–60, 62–64.
- CPU-offloaded checkpoints: 1–5, 7–9, 11–14, 26–27, 30–31, 33–35, 38, 49.

- NVMe-offloaded checkpoint: 21.
- Recomputed boundaries: 6, 17–19, 25, 37, 45–47, 53–55, 61.

In the unrestricted MILP, each interior layer boundary has four candidate actions: retain the activation in HBM, offload it to CPU DRAM, offload it to NVMe, or omit the checkpoint and recompute it during the backward pass. The final 27B run uses a restricted instance: fixed reference edges require $z_{21} = 1$, and the fixed reference GPU set requires $g_{21} = 0$. Thus, boundary 21 is optimized only between CPU and NVMe placement. Algorithm 2 summarizes the general search procedure.

A.2 MoE Note for GPT-OSS-120B

GPT-OSS-120B is a mixture-of-experts model: only a subset of experts is active for each token during the forward pass. This reduces active forward compute relative to a dense model with the same total parameter count. However, full fine-tuning still needs storage and optimizer states for all trainable expert parameters. The current LazyTrain scheduler is layer-level: it treats the MoE block as one layer-level object for placement and streaming. Expert-level placement, expert-aware optimizer-state tiering, and expert-aware communication overlap are natural follow-up work.

The main text visualizes a separate Qwen3.6-27B time-limit incumbent with an 8 GB CPU activation budget in Figure 4; it is not the final 15 GB-budget schedule summarized above. Additional solver case studies appear in Section G, with a direct GPT-OSS-120B JSON case in Section G.1.

B MILP Solution Procedure

LazyTrain instantiates the activation-placement and recomputation planner as a mixed-integer linear program (MILP). PySCIPOpt is used as the modeling interface and SCIP is used as the underlying solver. For the linear mixed-integer model in this work, SCIP’s exact search is appropriately described as *branch-and-cut*: a branch-and-bound tree over integer scheduling decisions, strengthened by presolve, LP relaxations, cutting planes, primal heuristics, and domain propagation.

B.1 Canonical MILP Form

Let $\mathcal{B} = \{0, \dots, L\}$ denote activation boundaries and $\mathcal{E} \subseteq \{(s, e) : 0 \leq s < e \leq L\}$ denote legal

Table 6: Representative solver-experiment outcomes. “Mandatory exposed” is the solver-reported parameter/gradient traffic that cannot be hidden by the configured compute windows. It is reported for diagnosis but subtracted as a constant baseline from the placement objective.

Model / setting	Status	Budget	Solver result	Interpretation
Qwen3.6-27B schedule				
Qwen3.6-27B dense; H800 MetaMathQA; batch 72	Optimal within restricted solve	Peak $\leq$ 70GB; CPU activation 15GB; NVMe activation 32GB	23.68B trainable parameters; activation 0.703GiB per checkpoint; 52 checkpoints: 30 GPU, 21 CPU, 1 NVMe, 13 recompute boundaries; objective 182.0015; exposed communication: activation/NVMe/-mandatory = 0/0/0ms	With recomputation edges and GPU checkpoints fixed, the final 27B schedule fits the 70GB HBM cap; the solver reports all activation movement hidden inside compute windows.
GPT-OSS-120B admission and placement				
GPT-OSS-120B MoE; 360GB CPU budget; hybrid 2% 8-bit Adam; batch 32 requested	Infeasible	Peak $\leq$ 70GB; CPU total 360GB	116.54B trainable parameters; CPU base 1321.38GiB; shortfall 961.38GiB	With normal Adam-style CPU states, host memory is the first blocker. Reducing batch size cannot fix optimizer-state memory.
GPT-OSS-120B MoE; 360GB CPU budget; full 8-bit Adam states; batch 32 requested	Infeasible	Peak $\leq$ 70GB; CPU total 360GB	CPU base 683.41GiB; shortfall 323.41GiB; no schedule admitted	Full 8-bit optimizer state helps substantially, but 360GB host memory is still not enough for this 120B configuration.
GPT-OSS-120B MoE; 2TiB CPU, 4GB runtime reserve, hybrid 2% 8-bit Adam; batch 32	Optimal solve	Peak $\leq$ 70GB; CPU total 2048GB	Static GPU 41.98GiB; activation budget 28.02GiB; CPU base 1321.38GiB; 37 checkpoints: all GPU activation; objective 0.00037; activation exposed 0ms; mandatory exposed 31259.54ms	Once CPU memory is relaxed, activation placement is easy; the bottleneck moves to unavoidable parameter/gradient streaming.
GPT-OSS-120B MoE; 2TiB CPU, 4GB runtime reserve, full 8-bit Adam states; batch 32	Optimal solve	Peak $\leq$ 70GB; CPU total 2048GB	Static GPU 41.98GiB; activation budget 28.02GiB; CPU base 683.41GiB; 37 checkpoints: all GPU activation; objective 0.00037; activation exposed 0ms; mandatory exposed 31259.54ms	8-bit states mainly reduce CPU memory; they do not remove the large parameter/-gradient traffic that must be overlapped.

recomputation segments. We collect the decision variables from Eq. (7) into

$$y = (x, z, g, c, n, d, h, q, r, \epsilon),$$

where $x_{s,e}$ selects recomputation segment (s, e) , z_i indicates whether boundary i is materialized, g_i, c_i, n_i place boundary i on GPU HBM, CPU DRAM, or NVMe, d/h assign CPU offload traffic to D2H/H2D windows, q/r assign NVMe write/read traffic, and ϵ records communication not hidden by compute windows. The resulting optimization problem has the standard MILP form:

$$\begin{aligned}
\min_y \quad & a^\top y \\
\text{s.t.} \quad & Ay \leq b, \quad Ey = e, \\
& \ell \leq y \leq u, \\
& x_{s,e}, z_i, g_i, c_i, n_i \in \{0, 1\}, \\
& d, h, q, r, \epsilon \geq 0.
\end{aligned} \tag{8}$$

Here the equality matrix E contains the checkpoint-path flow constraints and the unique-placement constraints. The inequality matrix A contains GPU/CPU/NVMe capacity constraints and per-window PCIe/NVMe bandwidth constraints. The linear objective $a^\top y$ is the weighted sum of recomputation cost, activation-induced communication exposure, and small tie-breaking terms. As in

Eq. (7), mandatory PCIe exposure that exists without activation offloading is subtracted as a constant baseline.

Relation to the runtime. The MILP is solved before training starts. Its output is a serialized schedule that the layer-streaming runtime consumes directly. Thus the solver is not on the critical path of every training step; it is an offline planning phase whose result fixes checkpoint boundaries, storage tiers, and communication windows.

B.2 Branch-and-Cut Search

At each node v of the search tree, SCIP solves the LP relaxation of Eq. (8), replacing binary restrictions by interval bounds $0 \leq x, z, g, c, n \leq 1$ and adding any branching decisions inherited from node v . For a minimization problem, this relaxation gives a valid lower bound $\theta(v)$ on all integer schedules in that subtree. If $\theta(v)$ is already worse than the best known feasible integer solution, i.e., the incumbent upper bound, the whole subtree is pruned.

Cutting planes tighten the LP relaxation. A cut is a valid inequality $\alpha^\top y \leq \beta$ that is satisfied by every feasible integer schedule but violated by the current fractional LP solution. Adding such cuts improves the bound without removing any legal LazyTrain schedule. If the tightened LP solution

Table 7: Boundary-21 decision trace for the final 27B schedule. Byte counts are decimal GB because they come directly from bandwidth windows.

Stage	Decision	Interpretation
Candidate placements		
General MILP option	$g_{21} = 1$	Keep boundary 21 in GPU HBM. This option is excluded from the final restricted solve because the reference GPU-checkpoint set fixes $g_{21} = 0$.
Final-solve candidate	$c_{21} = 1$	Offload boundary 21 to CPU; consumes DRAM and PCIe D2H/H2D windows.
Final-solve candidate	$n_{21} = 1$	Offload boundary 21 to NVMe; consumes SSD capacity, GPU-NVMe path, and PCIe/GDS windows.
General MILP option	$z_{21} = 0$	Omit boundary 21 and recompute across it. This option is excluded from the final restricted solve because the fixed adjacent reference edges require $z_{21} = 1$.
Solver-selected assignments		
Selected placement	$n_{21} = 1$; neighbors 20 and 22 stay on GPU	The solver selects NVMe for boundary 21 while preserving edges (20, 21) and (21, 22).
Selected read assignment	NVMe read: 0.198, 0.198, 0.0758, 0.283GB across windows 21, 22, 23, 63	The activation is read back in pieces so the SSD endpoint is used inside available compute windows.
Selected write assignment	NVMe write: 0.2345, 0.2345, 0.0515, 0.2345GB across windows 21, 22, 23, 41	The activation is written out in pieces; the solver reports 0ms exposed NVMe communication.

is still fractional, SCIP branches on one fractional integer variable, for example a placement variable n_{21} or an edge variable $x_{20,21}$, creating child subproblems with that variable fixed to 0 or 1.

Why this matters for LazyTrain. The branch-and-cut procedure is not a heuristic local search over checkpoint intervals. It systematically searches over checkpoint topology, activation tier placement, recomputation decisions, and communication-window assignments under hard memory and bandwidth constraints. The fixed MegaTrain heuristic can be represented as a feasible point in this space; LazyTrain asks SCIP to find a better feasible point and, when solved to optimality, to certify that no better point exists within the encoded model and solver tolerances. Individual runs may fix selected variables to a reference schedule. In the final Qwen3.6-27B run, the optimality certificate applies only after fixing the edges and GPU-checkpoint set from the LazyTrain - Hybrid 8-bit variant.

Tables 8–10 report the resource settings, training protocol, and configuration differences used to instantiate the MILP and interpret the 27B experiment.

C Experimental Configuration

D Experimental Server Environment

Table 11 and Table 12 report the concrete server environment used for the H800 experiments. These

Algorithm 2 SCIP Branch-and-Cut Search for the LazyTrain MILP

Input: MILP $\mathcal{M}$ from Algorithm 1

Output: Best feasible schedule y^* and final optimality gap, or infeasibility status

- 1: Presolve $\mathcal{M}$ to tighten bounds, remove redundant variables and constraints, and detect immediate infeasibility.
- 2: Initialize incumbent upper bound $U \leftarrow +\infty$ and global lower bound $L \leftarrow -\infty$.
- 3: Insert the root LP relaxation into the node queue.
- 4: **while** the node queue is not empty and the stop criterion is not met **do**
- 5: Select a node v and solve its LP relaxation.
- 6: **if** the LP is infeasible **then**
- 7: Prune v .
- 8: **else if** the node lower bound $\theta(v) \geq U$ **then**
- 9: Prune v by bound dominance.
- 10: **else**
- 11: Separate valid cutting planes and reoptimize the tightened LP.
- 12: **if** the LP solution is integer feasible **then**
- 13: Update y^* and U if its objective improves the incumbent.
- 14: **else**
- 15: Select a fractional binary scheduling variable and branch, creating two child nodes with that variable fixed to 0 and 1.
- 16: **end if**
- 17: **end if**
- 18: Update the global lower bound L from all open nodes.
- 19: **end while**
- 20: **return** incumbent schedule y^* and gap $|U - L| / \max\{1, |U|\}$.

Table 8: Hardware and scheduler resource parameters for the H800 experiments.

Item	Configuration
Accelerator	NVIDIA H800, 80GB HBM per single-GPU run.
Host	Dual Intel Xeon Gold 6448Y CPUs, 128 CPU threads, 2.0TiB DDR5 memory.
Local storage	Two local NVMe-backed data mounts, about 7TB each on the experiment node.
GPU–CPU link	PCIe Gen5 x16 node; the measured activation H2D/D2H rate used by the LazyTrain scheduler is 12GB/s per direction.
GPU–NVMe path	GDS/cuFile direct path in the final NVMe-aware run; measured NVMe read/write endpoint rates are 2.83/3.35GB/s.
HBM cap	LazyTrain schedules use a configured 70GB peak cap; the observed NVMe-aware training peak is 68.84GB.

Table 9: Common Qwen3.6-27B / MetaMathQA training configuration. The evaluation split is also used for periodic loss monitoring.

Item	Configuration
Model and data	
Model	Qwen3.6-27B; Qwen3.5-style text architecture (Qwen Team, 2026): 64 layers, hidden size 5120, 24 attention heads, 4 KV heads, intermediate size 17408. The scheduler computes 23.68B trainable parameters from this configuration.
Precision and attention	bfloat16 training with PyTorch scaled dot-product attention (SDPA).
Dataset	ModelScope MetaMathQA converted to a 70/30 split: 276,499 training examples and 118,501 evaluation examples.
Prompt format	Qwen3.5 non-thinking template; only response tokens contribute to the training loss.
Optimization and evaluation	
Batch and length	Per-step batch size 72, sequence length 1024, gradient accumulation 1.
Training length	3841 optimizer steps, equal to one pass over the 276,499-example training split with batch size 72.
Optimizer hyperparameters	Adam-style optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, learning rate 10^{-5} , weight decay 0.01, max grad norm 1.0, seed 42.
Evaluation	Initial evaluation on 0.1 of the evaluation split before training ($\approx 11,850$ examples), then evaluation on 10,000 sampled examples every 0.1 epoch; evaluation batch size 72, seed 42. Final exact-match evaluation uses the full evaluation split and the same non-thinking prompt template.
Safety checks	Non-finite gradients are skipped; training aborts after three consecutive non-finite-gradient events.

are reproducibility details recorded on the experiment server. They are separate from the scheduler resource model in Appendix C: that appendix reports the abstract capacities and bandwidths consumed by the MILP, while this section reports the actual operating system, devices, drivers, and Python package versions of the experiment server.

E Reproducibility Notes

Training protocol. The main 27B experiment fine-tunes Qwen3.6-27B on MetaMathQA with a 70/30 training/evaluation split, sequence length 1024, batch size 72, and one epoch. At this batch size, the training split contains 3841 optimizer steps. The protocol runs an initial evaluation and then evaluates every 0.1 epoch on 10,000 sampled evaluation examples; the reported final accuracy uses the full evaluation split for the final LazyTrain configuration.

Hardware checks. The server has eight NVIDIA H800 80GB GPUs, two Intel Xeon Gold 6448Y sockets, 128 CPU threads, 2.0 TiB host memory, and two local NVMe-backed data mounts; each reported run uses one H800. GPU memory and PCIe Gen5 x16 were confirmed with `nvidia-smi`; CPU and storage capacities were confirmed with standard system inspection tools. The NVMe read-/write endpoint values used by the scheduler are 2.83/3.35 GB/s. Full hardware and software configurations appear in Tables 11 and 12.

Reproducibility package. The accompanying release contains the scheduler implementation, training configurations, evaluation scripts, and solved schedules used by the reported setup. It is available at uploaded source code.

Table 10: Configuration differences among the compared Qwen3.6-27B variants. All use the same model, data split, batch size, and sequence length; the LazyTrain - MILP row is a partial experiment without final accuracy. A minus sign denotes removal of the named component from the complete system.

Variant	Optimizer	Scheduling policy	Runtime setting
MegaTrain baseline	DeepSpeed CPUAdam	MegaTrain-compatible fixed checkpoint interval 4; no optimized activation-placement solve.	Baseline execution setting.
LazyTrain	Hybrid 8-bit operator: DeepSpeed CPUAdam plus 2% CPU 8-bit AdamW state slice with block size 4096, together with fast gradient clipping.	Restricted GPU/CPU/NVMe solve with recomputation edges and GPU checkpoints from the LazyTrain - Hybrid 8-bit variant fixed: peak cap 70GB, CPU/NVMe activation budgets 15/32GB, and final placement counts 30/21/1.	Final NVMe-aware runtime setting.
LazyTrain - MILP	Same complete Hybrid 8-bit operator as the full system.	MILP scheduling disabled.	Partial experiment without final accuracy measurement.
LazyTrain - Hybrid 8-bit	DeepSpeed CPUAdam; both the CPU 8-bit AdamW state slice and fast gradient clipping are disabled.	MILP-selected activation schedule retained with the 70GB peak cap and PCIe bandwidth constraints.	Optimizer-side ablation setting.

Table 11: Experimental server hardware and system configuration.

Item	Measured configuration
Host system	
Operating system	Ubuntu 22.04.5 LTS, Linux kernel 5.15.0-113-generic, x86_64.
CPU	2× Intel Xeon Gold 6448Y sockets; 32 cores per socket; 2 threads per core; 128 logical CPUs. The CPU supports AVX-512, AVX-512 BF16, AMX BF16, and AMX INT8 instructions.
System memory	2.0TiB host DRAM; swap disabled.
Accelerators and interconnect	
GPU devices	8× NVIDIA H800 GPUs. Each GPU reports 81,559MiB HBM through <code>nvidia-smi</code> , i.e., the 80GB H800 class used for single-GPU experiments.
GPU driver and CUDA	NVIDIA driver 575.57.08; CUDA 12.9 reported by <code>nvidia-smi</code> .
PCIe capability	Each H800 reports PCIe Gen5 x16 as the maximum link capability. The LazyTrain scheduler uses a conservative 12GB/s per-direction activation-transfer bandwidth for CPU–GPU traffic.
GPU interconnect	The eight H800 GPUs are connected by NVLink. The reported experiments use one GPU and do not rely on inter-GPU communication.
Local storage	
Local storage	Root filesystem: 893.8GB RAID device. Two data mounts are each backed by a 7TB Samsung MZQL27T6HBLA NVMe SSD. The LazyTrain NVMe-aware offload path uses local NVMe storage.

F Broader Impact and Compute Resources

Broader Impact. The intended benefit of LazyTrain is to make large-model fine-tuning more accessible to groups with limited accelerator memory. Lowering the hardware barrier can help academic labs and smaller organizations run controlled experiments without relying exclusively on large shared clusters. The same capability can also reduce the friction of training models for harmful uses, so deployment should follow the same data governance, safety evaluation, and access control practices expected for the underlying model and dataset.

Compute Resources. The reported experiments use single H800 80GB and RTX 3090 24GB GPUs. The primary 27B experiment uses one H800 with CPU DRAM and local NVMe storage. The pa-

per reports per-run throughput and memory use rather than total project compute. Reproducing the complete 27B run requires one epoch over the 70% MetaMathQA training split at batch size 72 and sequence length 1024, corresponding to 3841 optimizer steps in the reported protocol.

G Visual Solver Case Studies

Table 6 and Table 7 report the detailed solver values. Figure 7 summarizes the same evidence as a visual progression: red marks the constrained state, blue the selected or strengthened configuration, and green the resulting system implication.

The top row makes the boundary-level solver output easier to read: the general model admits GPU, CPU, NVMe, and recomputation choices, whereas the final run fixes the recomputation path and GPU-checkpoint set from the LazyTrain - Hybrid 8-bit variant before selecting NVMe for boundary 21. In

Table 12: Software stack used by the LazyTrain experiments and runtime.

Component	Version / setting
Python	Python 3.12.13.
PyTorch stack	PyTorch 2.10.0+cu129; CUDA runtime 12.9; cuDNN 91701; Triton 3.6.0.
Training libraries	DeepSpeed 0.19.2; Transformers 5.13.0; Datasets 4.0.0; W&B 0.28.0; NumPy 2.5.1.
Optimization solver	PySCIPOpt 6.2.1 wrapping SCIP for the MILP scheduling solve.
Auxiliary CUDA/sequence packages	flash-linear-attention 0.5.1 (f1a); causal-conv1d 1.6.2.post1.
Compilation tools	GCC 11.4.0; NVIDIA nvcc 12.9.41.

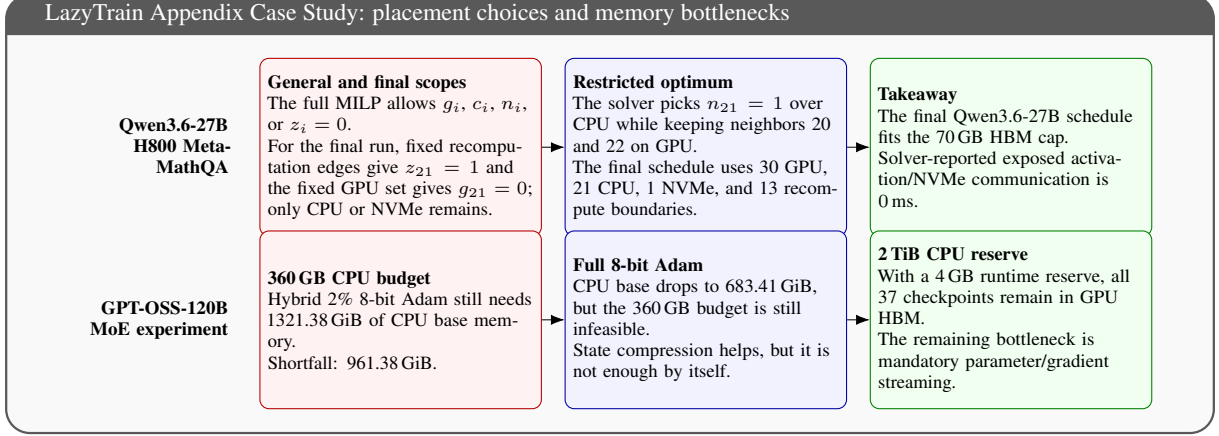


Figure 7: Appendix solver case study. Red boxes mark the bottleneck state, blue boxes mark the solver-selected or strengthened configuration, and green boxes mark the resulting takeaway. The top row distinguishes the general action space from the restricted Qwen3.6-27B boundary-21 solve; the bottom row follows the GPT-OSS-120B host-memory sweep.

this restricted experiment, the solver reports all activation movement hidden inside compute windows. A separate JSON-derived Qwen3.6-27B time-limit incumbent with a tighter 8 GB CPU activation budget appears in Figure 4 in the main text. The bottom row shows that host memory is a distinct constraint from activation placement: the 1321.38 GiB hybrid-state and 683.41 GiB full-8-bit-state requirements both exceed the 360 GB budget. A 2 TiB host budget makes the placement feasible but leaves mandatory parameter/gradient streaming as the remaining overlap problem.

G.1 GPT-OSS-120B Solver-Output Case

Figure 8 renders a second solver-output JSON for the GPT-OSS-120B Hugging Face configuration at batch size 72. This is a separate solver experiment from the batch-32 GPT-OSS results in Table 6: the JSON reports an optimal SCIP solve, materializes 34 activation boundaries, and recomputes the first three interior boundaries.

This GPT-OSS-120B case shows a different solver regime from the Qwen3.6-27B case. Under the 2 TiB host-memory experimental setting, the activation-placement decision is simple: no

CPU or NVMe activation offload is selected, and the solver uses only a short recomputation prefix. The objective value primarily reflects that recomputation. Mandatory parameter/gradient exposure is reported separately as a diagnostic baseline and is not charged to the placement objective.

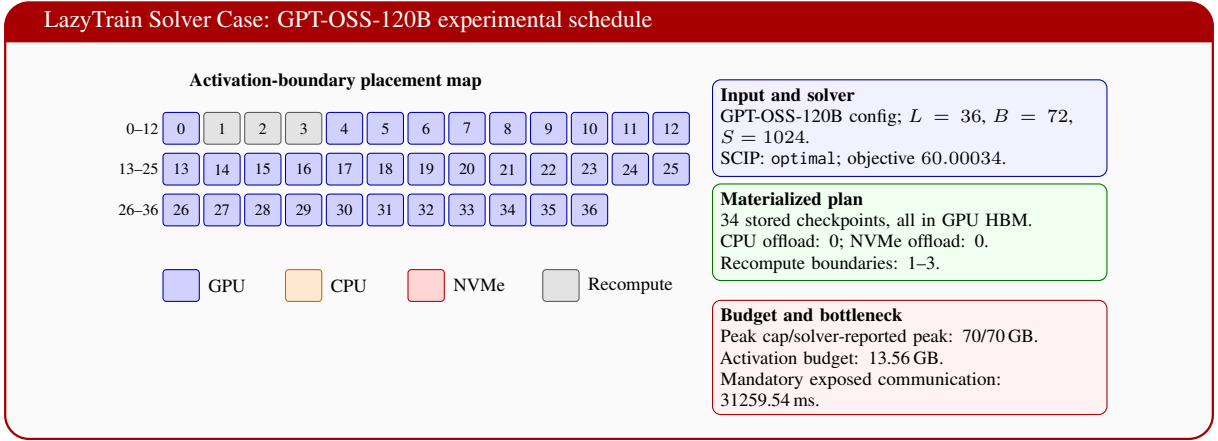


Figure 8: Boundary-wise JSON schedule map for the GPT-OSS-120B solver experiment at batch size 72. Each tile is one activation boundary from 0 to 36. The solver keeps every materialized activation checkpoint in GPU HBM and selects recomputation for boundaries 1-3; the JSON reports 0 ms exposed activation and NVMe communication, while mandatory parameter/gradient traffic remains exposed.